What Keeps a Generative Recommender Relevant? An Empirical Study of Reinforcement Fine-Tuning Against a Learned Reward Model

Daniil Antonov

ITMO University, Saint Petersburg, Russia

Draft, version 3 14 October 2026

Abstract

A recommender that generates a list item by item can be fine-tuned with reinforcement learning (RL) towards feedback that next-item training does not see. We study the least protected form of this idea, in which a learned model of user feedback is the only source of reward: a GPT-2 recommender is fine-tuned with a policy gradient to raise the consumed share of its items as a second GPT-2 predicts it, held near the pretrained recommender by a KL anchor, on a video log (KION) and a music log (Yambda-50M). Four instruments that the optimised model cannot influence judge the result: a reward model fitted on other users, the logged response to the recommended items that users went on to consume, the same lists scored for another user, and relevance split by the logged outcome. On KION the predicted share rises by 21.5% and the held-out model agrees, yet the logged share per recommended item does not change, and 99% of the gain is what any other user would give the same lists: the policy learned which titles everybody finishes. A reward that is neutral to how easy an item is to finish removes this drift and leaves RL little to gain. The anchor weight is not a comparable quantity: at one weight a leave-one-out baseline moves the policy seven times further than a value model. The KL distance is comparable: at equal distance the two estimators give the same policy, and crediting an item with its own reward only is no worse than crediting it with the rest of the list. Re-ranking the pretrained recommender's candidates matches or beats RL at every moderate operating point, also for a reward that depends on the list as a whole. The one reward term that improves logged outcomes is itself read from the log. All results are offline and single-seed.

Keywords: generative recommendation, sequential recommendation, reinforcement learning, reward model, reward hacking, offline evaluation.

1 Introduction

Sequential recommenders built as autoregressive transformers over item or semantic identifiers [1], [2], [3], [4], [5] are trained to continue logged sequences. The log records what users did, not what served them: in the two logs studied here, 36% of music listens stop before half of the track and 52% of video titles are abandoned before half of their length. A recommender that generates a list item by item is also a policy that takes a sequence of decisions, so reinforcement learning (RL) is the natural tool for moving it from "what comes next" towards "what will be received well" [6], [7], [8], [9].

Where the reward comes from decides how safe that step is. Two recent studies that report gains without a loss of relevance both keep the reward tied to the log. Petrov and Macdonald [8] fine-tune GPTRec with PPO on its own generated lists, with a reward that contains NDCG against the user's true next item. Matveev et al. [9] train a retriever with off-policy REINFORCE on logged sessions, with logged feedback as the reward, and use a learned feedback model only to evaluate. The third option is to let a learned model of user feedback *be* the reward. It is attractive because such a model scores any item for any user, so the policy can be trained on its own generations and for signals that the log holds only for the items users chose. It is also the option with the fewest protections: a policy optimised against a learned model drifts to where the model is wrong [10], [11], [12], [13], and in recommendation the drift takes the form of popularity and filter-bubble effects [14], [15], [16].

This paper studies the third option in its barest form. A GPT-2 [17] next-item recommender is fine-tuned with a policy gradient to raise the share of an item that a second GPT-2, the *reward model*, predicts the user

will consume. Nothing in this reward asks for relevance; a KL penalty towards the pretrained recommender, the *anchor*, is the only term that does. We ask five questions.

RQ1 Does fine-tuning raise the reward, and is the gain real outside the model that was optimised?
RQ2 Is relevance preserved, do users still receive different lists, and which mechanism preserves either?
RQ3 Does the policy use the fact that a list is a sequence, and what does the reward of a generated list depend on?
RQ4 Can other signals serve as rewards: retention, a share that is neutral to item length, a reward for the whole list?
RQ5 What is the trade-off between reward and relevance, and does reaching it take RL?

Every reward gain in this setting is reported by a model. The pipeline therefore carries instruments that do not depend on the model being optimised, and the study is organised around what they show. Our contributions are the following.

- **A protocol for judging RL against a learned reward offline.** A second reward model fitted on a disjoint half of the users; the logged response to those recommended items that the user went on to consume, which no model stands behind; the same lists scored for another user, which separates what the policy learned about the user from what it learned about items; and relevance split by the logged outcome. Operating points are selected on validation requests only (Section 3.5, Section 4).
- **The gain is confirmed by a second model and by nothing else.** On KION the predicted consumed share rises by 21.5% and a reward model trained on other users agrees. The logged share per recommended item does not move (+0.6%, interval $[-0.6, +1.7]$), the logged share relative to what is usual for the item falls, and 99% of the predicted gain is what other users would give the same lists. Re-ranking candidates by an item's mean share in the training log, with no reward model, reaches the same point. A reward that pays for the share *relative to the item's usual* removes the drift towards easily finished items and leaves the policy almost nothing to gain (Section 5.2, Section 5.5, Section 5.7).
- **What the anchor constrains.** The weight of the anchor is measured against standardised advantages and has no meaning across estimators: at the same weight a leave-one-out baseline moves the policy 7 to 15 times further from the pretrained recommender than a learned value model. A budget on the KL divergence is comparable. At an equal budget the two estimators arrive at the same policy, and a policy credited with each item's own reward only, or trained on one-item episodes, is at least as relevant as one credited with the whole rest of the list: we find no evidence that the policy gradient uses the sequential structure of a list. PPO's clipping hardly took part (Section 5.3, Section 5.4).
- **Reaching the trade-off does not take RL.** Re-ranking the 100 best candidates of the pretrained recommender by the same reward model reaches the frontier of RL without any training, and raises the logged share where RL does not. This also holds for a reward that depends on the list as a whole, where a greedy construction is not optimal in principle. The reward term that improves logged outcomes under RL is logged relevance, and on a log with repeated consumption it does so by replaying the user's history (Section 5.6, Section 5.7).

All results are offline, and every configuration was trained with one seed. Section 6 separates the claims that offline evidence settles from those that need an online experiment, and Section 7 lists what remains open.

2 Related Work

Generative recommenders and RL. GRU4Rec [18], SASRec [1] and BERT4Rec [19] made next-item prediction the default formulation of sequential recommendation; generative recommenders use the same models autoregressively, over items [2] or semantic identifiers [3], up to industrial scale [4], [5], [20]. We keep the simplest member of the family so that the effects of fine-tuning are not confounded with tokenisation. RL enters recommendation through off-policy policy gradients [6], slate values [7], [21] and RL heads on supervised models [22], [23]. The two studies we build on sit at opposite ends: on-policy PPO over the recommender's own generations with a reward read from the true next item [8], and off-policy REINFORCE over logged sessions [9]. Learned models enter rewards when the wanted signal is sparse [24] and replace the user altogether in simulators [25], [26], [27]; ours is a simulator learned from the log the recommender was trained on.

Learned rewards and their exploitation. Model-based offline RL trusts a model only where data supports it [10], [11], [28]. For language models fine-tuned against reward models, a "gold" reward first rises and then falls while the proxy keeps rising [12]; ensembles mitigate this [29], and reward hacking has a formal treatment [13]. Our held-out reward model plays the part of the gold model, and we find that it is not gold enough: two models trained on the same kind of data share what they have learned about items. In recommendation, conservative offline RL amplifies the Matthew effect [14] and feedback loops degenerate [15], [16].

Constraints and rewards. TRPO and PPO bound each update relative to the previous iterate [30], [31]; the adaptive-penalty variant of PPO adjusts a KL coefficient towards a target divergence, the rule that our KL budget applies to the pretrained policy. Penalising divergence from a *pretrained* model is KL-control [32], [33], used when language models are fine-tuned from human preferences [34], [35] and as behaviour regularisation in offline RL [36]. Consumption time is confounded by item duration [37], [38]; we observe the confounding from the policy's side and remove it with a within-item quantile. Long-term engagement and retention have been optimised directly [39], [40] and through proxies [41], [42]. The diversity of a list is an objective that no single item can earn; maximal marginal relevance [43] is its greedy optimiser and our reference for a list-dependent reward.

Generation and evaluation. Feeding a model its own outputs takes it away from the inputs it was trained on [44], [45], and spending inference compute on several candidates that a learned model judges is the recommender form of test-time scaling [9], [46]. Next-item protocols hide deficiencies of RL recommenders [47], and off-policy estimators such as the doubly robust one [48] need logged propensities, which neither log provides. Results selected and judged by the same feedback model are not independent evidence [9], and baselines need one protocol [49].

3 Method

Figure 1 shows the pipeline: a recommender is pretrained on next-item prediction, two reward models are fitted on disjoint halves of the users, and the recommender is fine-tuned on the reward that one of them predicts for its own generated lists.

3.1 Setting and models

A log is a set of events, each with a user, an item i_t, a time and the *consumed share* $\rho_t \in [0, 1]$: the share of the track that was played or of the title that was watched. A recommender is a distribution $\pi_\theta(j \mid h)$ over the next item given the history h; π_0 is the recommender before fine-tuning, and a recommendation is a list $a_{1:K}$ with $K = 10$.

Base recommender. A GPT-2 decoder [17] over item tokens with tied input and output embeddings, trained with the full-softmax next-item cross-entropy. A list is obtained by *ranking* (Top-K: the catalogue is scored once) or by *generation* (Next-K: each chosen item is appended to the history as if the user had consumed it, and no item repeats within a list) [2]. Fine-tuning changes the second mode, in which the recommender is a policy over a sequence of decisions.

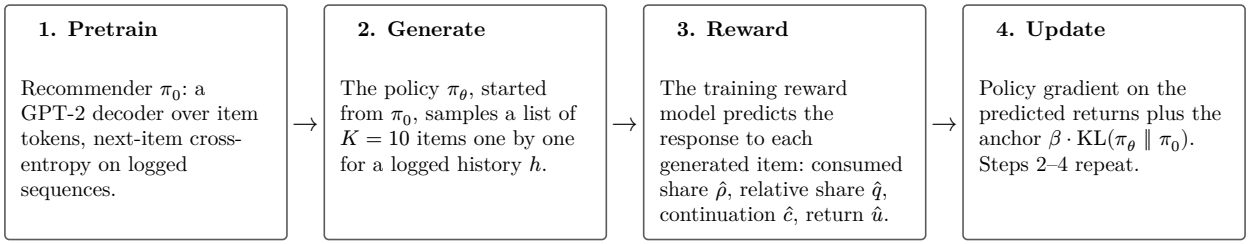


Figure 1: The pipeline. The reward model of step 3 is the only source of reward during fine-tuning; the instruments in the dashed box never influence the policy.

Reward model. A second GPT-2 decoder of the same size. The token at position t carries the item i_t together with the consumed share of the *previous* item in ten buckets, so the state at position t has seen the candidate item and the whole history with its responses, but not the response it predicts. A generated item can thus be scored before any outcome for it exists. Five heads read the state, each trained with binary cross-entropy: the consumed share $\hat{\rho}$ (against ρ as a soft label); the *relative share* $\hat{q}$ defined below; the probability $\hat{c}$ that the session continues (the user's next event follows within a gap g); the probability $\hat{u}$ that the user returns (a new session starts within a window w); and a choice head, which an earlier iteration of the study found useless as a relevance signal and which no reward here uses. A retention outcome counts as known only if its whole waiting time lies inside the observed data.

Relative share. The consumed share pays for items that are easy to finish, whoever receives them: a film over a series, a short track over a long one. To separate the user's response from the item's, each event also gets its mid-rank quantile among the training-period events of the same item,

$$q_t = \frac{\#\{\rho' < \rho_t\} + \frac{1}{2}\#\{\rho' = \rho_t\}}{n_{i_t}}, \tag{1}$$

where ρ' runs over the n_{i_t} training-period consumed shares of item i_t. Its mean is exactly one half for every item, so no item is better than another on average, and a policy can raise it only by matching items to the users who take to them more than others do.

Two copies. Two reward models are trained on disjoint halves of the users. The *training* reward model produces the rewards during fine-tuning. The *held-out* reward model never influences the policy and only scores its lists.

3.2 Reinforcement fine-tuning

One episode is the generation of one list for a history sampled from the training period. The state is $s_k = (h, a_{1:k-1})$, the action is the next item a_k, the reward r_k is predicted by the training reward model (by default $r_k = \hat{\rho}_k$), and the return is $G_k = \sum_{l \geq k} \gamma^{l-k} r_l$. The reward model is the environment. To score the k-th item it needs the responses to the items generated before it, which nobody observed; by default it is fed its own expectation $\hat{\rho}$ for them, and Section 5.4 compares this with sampling a response.

The policy gradient $\mathbb{E}\left[\sum_k A_k \nabla_\theta \log \pi_\theta(a_k \mid s_k)\right]$ is estimated with one of two advantage estimators. The default uses a learned *value model* V_φ, a copy of the recommender's transformer with a scalar head, and generalised advantage estimates [50]; each batch of lists is used for four gradient steps on the clipped surrogate of PPO [31]. The alternative is a *leave-one-out* baseline: several lists are sampled for each history, and the baseline of a list is the mean return of the others [51], [52]. Advantages are standardised to zero mean and unit variance within a batch in both cases. The clipping of the surrogate touched at most 0.3% of sampled items in any run that did not collapse (Section 5.3), so we do not attribute anything to PPO: in effect the default is an actor–critic policy gradient with several steps per batch.

Anchor and KL budget. The loss adds a penalty on the divergence from the frozen pretrained recommender, computed exactly over the catalogue at every position:

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PG}}(\theta) + \beta \cdot \mathbb{E}_k[\mathrm{KL}(\pi_\theta(\cdot \mid s_k) \parallel \pi_0(\cdot \mid s_k))]. \tag{2}$$

Since the default reward has no relevance term, β is the only thing that asks the policy to stay relevant. The penalty is weighed against standardised advantages, so the same β is a different constraint under another estimator, reward or dataset. A *KL budget* δ fixes the constraint itself: after every iteration the weight is multiplied by $\exp\left(\eta\left(\overline{\mathrm{KL}}/\delta - 1\right)\right)$, with $\eta = 0.03$ and the mean divergence $\overline{\mathrm{KL}}$ of the iteration, so that the policy settles at the distance δ in nats per generated item.

Credit along the list. Two ablations test whether fine-tuning uses the sequential structure of a list. With $\gamma = 0$ the return is the reward of the item itself. With *one-item episodes* a single item is sampled per episode, so the policy never learns from a context that contains generated items; at evaluation its lists are still generated item by item.

3.3 Rewards

All rewards are instances of

$$r_k = w_\rho(\hat{\rho}_k - b_k) + w_q\hat{q}_k + w_c\hat{c}_k + w_u\hat{u}_k + w_F\frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)} + \frac{w_D}{K-1}\sum_{l<k} d(a_k, a_l). \tag{3}$$

The default is $w_\rho = 1$ and everything else zero. The *relative-share* reward is $w_q = 1$ alone, and the retention rewards use w_c and w_u. The fifth term is the *logged-relevance* reward of [8]: for a history from the training period the user's real continuation F is known, and the term sums to DCG@K over the list. A baseline b_k makes the reward *personal*: it subtracts what the same reward model predicts for the same list shown to another user, or the item's mean share in the training period. The last term makes the reward depend on the *list as a whole*: d is the cosine distance between the embeddings of two items in π_0, and over a list the term sums to $w_D K/2$ times its mean pairwise distance. An item's worth then depends on its neighbours, and scoring items one at a time no longer finds the best list.

3.4 Alternatives to on-policy RL

Four methods raise the same rewards without generating lists in a training loop. **Liked-only fine-tuning** continues next-item training for one epoch on targets with $\rho \geq 0.5$; it uses logged feedback and no reward model. **Off-policy REINFORCE** [9] raises the log-probability of every logged item in proportion to the discounted logged reward that followed it in the session, with importance weights against π_0 capped at κ. **Re-ranking** takes the 100 best candidates of π_0 and keeps the K with the highest $r(j \mid h) + w \log \pi_0(j \mid h)$, where r is the reward predicted for j as the next item after the real history; nothing is trained, and w plays the part of the anchor. For the list reward it becomes greedy maximal marginal relevance [43]: the list is filled one position at a time, and the distance to the items already picked is added to the score. **Best of n lists** samples n lists from π_0 and serves the one with the highest predicted return, the test-time scaling of [9].

3.5 Instruments

For evaluation, F is the set of the next $M = 20$ items the user really consumed after h.

- **Held-out reward model.** A gain that the reward model of the other half of the users does not confirm is exploitation of the errors of the training model.
- **Logged response.** A recommended item that is in F, a *hit*, comes with the response the log recorded, and no model stands between that response and the recommender. We report the number of hits per list, their mean logged share, and two quantities per recommended item in which an item that was not hit counts as nothing: the *logged share* $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\rho(a_k)$ and the *logged surplus* $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\big(q(a_k) - \frac{1}{2}\big)$, which is positive only if users took to the recommended items more than people usually take to those items. Hits are a selected sample of the recommendations: these numbers say how the items the user did reach were received, not how the rest of the list would have been.
- **Another user.** A list generated for a user with history h is also scored for a user with another history h'. The difference $R_{\text{own}} - R_{\text{other}}$ is the part of the predicted response that depends on who receives the list; it is zero for a recommender that ignores the user. The gain of a method over π_0 splits into the gain other users would give the same lists and the rest, whose share of the total is the *personal share of the gain*.
- **Relevance by outcome.** Recall and NDCG against F and against its parts: *liked* ($\rho \geq 0.5$) and *skipped* items; items consumed *above* and *below usual* (ρ against the item's mean share in the training period), which asks the same question without favouring items that are easy to finish; and, where repetition is allowed, items that are *repeats* from the visible history and *new* ones.
- **Concentration.** Catalogue coverage (the share of items that appear in at least one list) and the expected overlap between the lists of two users (*pair overlap*).

4 Experimental Setup

Data. Yambda-50M [53] is a listening log of Yandex Music: each event has the track, a timestamp, the played share, the track length and a flag telling whether the user chose the track or a recommender served it. KION [54] is a log of an online cinema with one row per user and title: the date of the last viewing and the watched share of the title. Both logs are split along one time axis: the last 7 days are the test period, the 7 days before them the validation period, the rest the training period; the catalogue and all item statistics come from the training period only (Table 1). The consumed share is bimodal in both logs, so the reward

Table 1: The two processed logs.

	KION	Yambda-50M
Users	256,407	8,711
Items in the catalogue	7,992	50,000
Events, millions (training / validation / test)	3.51 / 0.19 / 0.18	35.0 / 1.13 / 1.08
Events per user, median	10	2,638
Mean consumed share	0.50	0.65
Events consumed at least half	48%	64%
Events that continue the session / are followed by a return	42% / 71%	95% / 92%
Events the user chose (not served by a recommender)	not recorded	51%
Test requests	59,997	7,593

model is in effect a skip classifier. On KION the share depends strongly on the length of the title: films are watched to 0.5 on average, and the longest tenth of the catalogue, which is series, to 0.22, because the share is taken of the whole series. A reward built on the share prefers films before any model is involved. On Yambda the session continues if the user has another event within 30 minutes and the user returns by starting a new session within a day; KION only has dates, so a session is a calendar day and a return is a viewing on another day within a week.

Models and training. The recommender and the reward models have 4 layers, width 256 and 4 attention heads, with windows of 48 events on KION and 128 on Yambda, trained with AdamW [55] and stopped by the validation loss. Re-recommending items from the history is allowed on Yambda, where repeated listening is normal, and blocked on KION, where every user–title pair occurs once; every method obeys the same constraint. Fine-tuning samples one list for each of 512 histories per iteration ($\gamma = 0.99$, $\lambda = 0.9$, clipping threshold 0.2, learning rates 10^{-5} for the policy and 10^{-4} for the value model, which is trained alone for the first 50 iterations). The leave-one-out estimator samples 4 lists for each of 128 histories, so both see as many lists per iteration. The *main run* has $\beta = 0.3$ and 2,000 iterations; every other fine-tuning run has 1,000.

Baselines. The most popular items, a replay of the user's own history (most played first), a first-order Markov chain, GRU4Rec [18], SASRec with its original loss [1] and BERT4Rec [19]. The neural baselines take the width, depth, dropout and window length of the recommender and are stopped by their own validation loss; none was tuned.

Evaluation. Each test user with a history makes one request at the start of the test week: the last 38 events on KION (118 on Yambda) are the history and the next at most 20 events are the ground truth. All quantities are measured on generated lists of ten. The change from the base to the fine-tuned recommender is reported with a 95% interval from a paired bootstrap over users, which measures the noise of the user sample and not the noise between training runs.

Selection. No model, hyperparameter or operating point was chosen on the test period within this protocol. Model sizes and learning rates were fixed before the study. Sweeps are reported in full, and where one point of a sweep has to be named we name the run with the highest held-out predicted share on *validation* requests among the runs that keep the validation NDCG@10 within 5% of the base recommender's. The weight $\beta = 0.3$ of the main run is the exception: it was carried over from earlier iterations of this study, in which test metrics of anchor sweeps had been seen, so it is an illustration and not a selected optimum. By the validation rule the selected weights are $\beta = 1$ on KION and $\beta = 0.05$ on Yambda (Table 15).

Sessions and repeatability. The KION results come from one session of 49 fine-tuning runs (6.1 hours on one 96 GB GPU). The Yambda results come from two: a first session of 12 runs (8.1 hours, of which 4.4 go to the base models and the baselines) and a continuation that reused its models and runs and trained 13 more (4.2 hours; Table 14). The continuation re-evaluated every reused run and reproduced the numbers of the first session to four decimals. The Yambda grids are still narrower than KION's (Table 13). Every configuration was trained with one seed. A second KION session with the same seed reproduced the base recommender exactly and the 26 fine-tuning runs the two sessions share with a median difference of 0.0003 in NDCG@10 and 0.001 in predicted share (at most 0.003 and 0.007): the noise of the hardware, a lower bound on the variation between seeds.

Table 2: Baselines on generated or ranked lists of ten for the test requests. The last two rows are the recommender that is fine-tuned, ranked and generated. On KION items of the history are blocked, so every hit is a new item.

	KION			Yambda			
	NDCG@10	**Recall@10**	**Coverage**	**NDCG@10**	**on repeats**	**on new**	**Coverage**
Most popular	0.0903	0.1303	0.004	0.0186	0.0193	0.0115	0.000
Replay of the history	0.0903	0.1303	0.004	0.1873	0.3722	0.0000	0.519
Markov chain	0.0959	0.1394	0.466	0.0550	0.0934	0.0139	0.457
GRU4Rec	0.1215	0.1721	0.544	0.1116	0.1706	0.0298	0.451
SASRec	0.1082	0.1580	0.335	0.0743	0.1070	0.0242	0.367
BERT4Rec	0.1290	0.1811	0.427	0.1036	0.1717	0.0232	0.461
GPT-2, ranked	0.1220	0.1726	0.574	0.1138	0.1765	0.0315	0.466
GPT-2, generated	0.1173	0.1620	0.495	0.1172	0.1865	0.0261	0.519

5 Results

In the tables, "Pred. share" is the consumed share of the generated lists as the held-out reward model predicts it, and "Logged" and "Surplus" are the logged share and the logged surplus per recommended item of Section 3.5.

5.1 Base models

Recommender. On KION the recommender is mid-table (Table 2): level with GRU4Rec, 13% above SASRec and 5% below BERT4Rec for ranked lists, and popularity is a strong floor. Generating a list item by item costs 4% of NDCG@10 against ranking. On Yambda the recommender is the best of the trained models, and all of them are far below a replay of the user's own history: NDCG@10 0.187 against 0.117. The split into repeats and new items shows where the trained models stand. Replay scores only on repeats; on tracks the user has not played in the visible history, the recommender is the best ranked model (0.032) and popularity reaches a third of it. Every statement below about relevance on Yambda is relative to this recommender, and the split is reported wherever a method could gain by replaying.

Reward model. On held-out events the reward model predicts the consumed share with AUC 0.755 on KION and 0.868 on Yambda (Table 3), against 0.656 and 0.607 for the item's mean share and 0.673 and 0.798 for the user's recent mean. The two copies agree (correlation of predictions 0.94 and 0.95). The right half of the table measures what a signal offers a recommender that chooses among its own candidates: how much it varies between the 100 candidates of one user, against how much its mean varies between users. On Yambda the predicted share varies five times less between one user's candidates than between users: it is mostly a property of the listener, which leaves a policy little to act on. Retention is predictable on KION (AUC 0.72 for return) and barely on Yambda (0.61), and on both logs it varies several times less between candidates than between users.

5.2 What fine-tuning changes

Table 4 compares the main run with the base recommender.

KION: the models agree, the log does not. The training reward model reports a gain of 20.3% and the held-out one 21.5%: by the standard check for exploitation the gain is real. Relevance falls by 10.7% overall and by 18.2% on skipped titles, and it is kept on liked ones (+1.3%), which looks like a policy that has learned to avoid what users abandon. The other instruments disagree. The policy recommends fewer titles that the user went on to watch (hits −8.3%), and those are finished more often (+9.8%) because they are titles that everybody finishes more often: the mean duration of a recommended title falls from 5.7 to 3.4

Table 3: The held-out reward model on test events (AUC; the share heads against the outcome thresholded at one half), and the standard deviation of each prediction between the 100 candidates of one user ("candidates") and of its mean over candidates between users ("users").

	KION			Yambda		
	AUC	**Candidates**	**Users**	**AUC**	**Candidates**	**Users**
Consumed share $\hat{\rho}$	0.755	0.144	0.153	0.868	0.052	0.250
Relative share $\hat{q}$	0.702	0.042	0.091	0.833	0.039	0.145
Session continues $\hat{c}$	0.657	0.050	0.093	0.603	0.006	0.016
Return $\hat{u}$	0.722	0.033	0.125	0.611	0.009	0.094

Table 4: The main fine-tuning run ($\beta = 0.3$) against the base recommender. Changes are relative, in percent, with a 95% paired-bootstrap interval over users where the quantity is defined per user.

	KION			Yambda		
	before	**after**	**change, %**	**before**	**after**	**change, %**
Relevance against the user's logged future						
NDCG@10, all next items	0.1173	0.1047	−10.7 [−11.6, −9.8]	0.1172	0.1183	+0.9 [−0.7, +2.5]
NDCG@10, liked items	0.1214	0.1229	+1.3 [+0.1, +2.4]	0.1005	0.1029	+2.4 [+0.6, +4.3]
NDCG@10, items consumed above usual	0.1261	0.1142	−9.4 [−10.6, −8.2]	0.0998	0.1023	+2.5 [+0.7, +4.3]
Recall@10, skipped items	0.1382	0.1130	−18.2 [−19.6, −16.8]	0.0724	0.0735	+1.5 [−1.7, +4.6]
Predicted consumed share of the list						
Training reward model	0.417	0.502	+20.3 [+20.1, +20.4]	0.594	0.608	+2.2 [+2.1, +2.4]
Held-out reward model	0.455	0.553	+21.5 [+21.4, +21.7]	0.599	0.609	+1.7 [+1.5, +1.9]
Held-out, list shown to another user	0.442	0.539	+22.0 [+21.9, +22.2]	0.498	0.508	+2.0 [+1.7, +2.3]
Held-out, relative to the item's usual	0.529	0.524	−1.0 [−1.1, −1.0]	0.469	0.468	−0.3 [−0.4, −0.1]
Logged response to the recommended items the user went on to consume						
Such items per list	0.359	0.329	−8.3	0.943	0.961	+1.9
Their logged share	0.481	0.528	+9.8	0.710	0.713	+0.5
Logged share per recommended item	0.0173	0.0174	+0.6 [−0.6, +1.7]	0.0669	0.0686	+2.5 [+0.3, +4.6]
Logged share above the item's usual, per item	0.0012	0.0005	−61.9 [−69.4, −54.4]	0.0033	0.0032	−3.0 [−15.2, +9.1]
Lists						
Catalogue coverage	0.495	0.449	−9.2	0.519	0.506	−2.6
Overlap between two users' lists	0.2092	0.2509	+20.0	0.0014	0.0020	+42.9
Mean duration of an item, minutes	339.1	205.7	−39.3	3.4	3.3	−3.0

hours, away from series. The two effects cancel, and the logged share per recommended item is unchanged (+0.6%, $[-0.6, +1.7]$). Measured against what is usual for each title, users took to the recommended titles less than before: the logged surplus falls by 62%, and relevance on titles watched above usual falls by 9.4%, almost as much as overall. The predicted relative share agrees (−1.0%). Finally, the same lists shown to *other* users are predicted to earn the same gain (+22.0%).

Yambda: little to gain. The same weight moves the policy less than half as far (KL 0.037 against 0.092 nats per item) and changes little: +1.7% of predicted share, with NDCG@10 inside its interval and a small gain on liked tracks. The logged share per recommended item rises by 2.4% ($[+0.3, +4.6]$) and the surplus does not move; the policy has shifted slightly towards popular tracks (mean popularity +13%). The value model explains 78% of the variance of returns here, against 49% on KION: the reward is mostly determined by the listener, as Table 3 predicted.

Table 5: Anchor sweep with the value-model estimator, 1,000 iterations per run. KL is the divergence from the base recommender at the end of fine-tuning, in nats per generated item; "Training model" is the share predicted by the reward model that produced the rewards.

	KL	**NDCG@10**	**on liked**	**Training model**	**Pred. share**	**Logged**	**Coverage**	**Overlap**
KION								
Base recommender	0.000	0.1173	0.1214	0.417	0.455	0.0173	0.495	0.209
$\beta = 1$	0.015	0.1154	0.1245	0.445	0.487	0.0178	0.479	0.219
$\beta = 0.5$	0.040	0.1120	0.1256	0.470	0.517	0.0180	0.458	0.237
$\beta = 0.3$	0.080	0.1043	0.1233	0.500	0.553	0.0175	0.436	0.251
$\beta = 0.2$	0.140	0.0983	0.1193	0.521	0.576	0.0168	0.413	0.266
$\beta = 0.1$	0.590	0.0714	0.0910	0.616	0.655	0.0114	0.287	0.316
$\beta = 0$	6.215	0.0005	0.0008	0.841	0.687	0.0002	0.008	0.983
$\beta = 0.3$, 2,000 iterations	0.092	0.1047	0.1229	0.502	0.553	0.0174	0.449	0.251
Yambda								
Base recommender	0.000	0.1172	0.1005	0.594	0.599	0.0669	0.519	0.001
$\beta = 0.3$	0.029	0.1176	0.1018	0.604	0.608	0.0681	0.505	0.002
$\beta = 0.1$	0.102	0.1158	0.1008	0.615	0.617	0.0668	0.479	0.004
$\beta = 0.05$	0.286	0.1115	0.0980	0.629	0.629	0.0653	0.433	0.006
$\beta = 0.03$	4.391	0.0032	0.0033	0.871	0.774	0.0017	0.015	0.766
$\beta = 0$	6.926	0.0008	0.0009	0.861	0.810	0.0003	0.001	0.999
$\beta = 0.3$, 2,000 iterations	0.037	0.1183	0.1029	0.608	0.609	0.0686	0.506	0.002

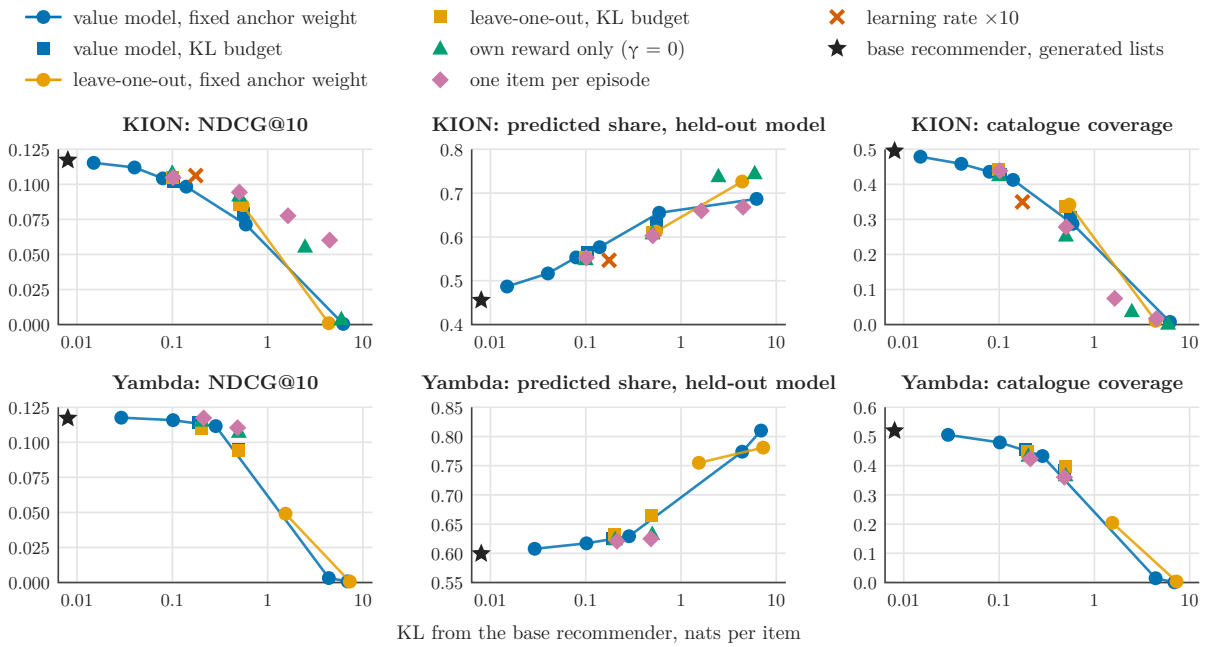


Figure 2: Every fine-tuning run of the default reward against the distance it ended at from the base recommender (logarithmic axis; the base recommender is drawn at the left edge). Runs that differ in the advantage estimator and in how the constraint is set lie on one curve. Runs with myopic credit lie on it at the KL budgets and, at a fixed weight, far to the right, above it in relevance and below it in coverage. For one-item episodes the distance is that of the first position only.

5.3 The anchor: a weight is not a distance

Without an anchor the policy collapses, and the held-out model partly applauds. At $\beta = 0$ nearly every user receives the same list on both datasets (pair overlap 0.98 and 1.00, Table 5). The training reward model reports a share of 0.84 on KION and the held-out one 0.69: the gap is exploitation, but the held-out value is still 51% above the base recommender. A second model trained on the same kind of data is fooled in the same direction, only less. The model-free instruments are unambiguous: NDCG@10 and the logged share are near zero.

The trade-off is smooth on KION and a cliff on Yambda. On KION relevance falls and the predicted share rises steadily as the anchor weakens. The logged share is highest at $\beta = 0.5$ (+4%) and is below the base recommender from $\beta = 0.2$ on. By the validation rule the selected run is $\beta = 1$: 1.6% below the base recommender in NDCG@10, 7% above in predicted share, 3% above in logged share. On Yambda relevance and predicted share move by a few percent down to $\beta = 0.05$ (KL 0.29: NDCG@10 −4.9%, predicted share +5.0%), and $\beta = 0.03$ collapses the policy (KL 4.4): the cliff lies between these two weights, and the validation rule selects $\beta = 0.05$.

The weight means different things under different estimators; the distance does not. Table 6 changes only the baseline of the policy gradient, with the number of lists per iteration, the learning rate and the passes over a batch held equal. At $\beta = 0.3$ on KION the leave-one-out estimator ends 6.8 times further from the base recommender than the value model (KL 0.55 against 0.08), and at $\beta = 0.1$ it collapses where the value model does not; on Yambda the factor at $\beta = 0.1$ is 15. Under a KL budget the two estimators end at nearly the same distance and on the same frontier, while the weights that hold the budget differ by a factor of 2.3 to 3.3. Where one of them settles slightly further away (on KION the value model, KL 0.104 against 0.099 and 0.56 against 0.50; on Yambda at the smaller budget the leave-one-out run, 0.200 against 0.189), it is correspondingly less relevant at a higher predicted share. The budget runs lie on the frontier of the weight sweep (Figure 2). On Yambda the budget also controls what the weight cannot: budgets of 0.2 and 0.5 nats end at almost the same weight (0.066 and 0.067). In this region the distance at which a run settles depends on more than its weight, so a fixed weight hardly selects an operating point there, while a budget does ($\delta = 0.5$: NDCG@10 −19%, predicted share +11%).

One reading of the factor is that standardising the advantages gives the policy-gradient term the same scale under both estimators while the share of that scale that is signal differs: a leave-one-out advantage

Table 6: The value model against the leave-one-out baseline at equal anchor weights and at equal KL budgets. "β at end" is the weight the run finished with. "Clipped" is the share of sampled items whose probability ratio left PPO's clipping range.

	β at end	KL	NDCG@10	Pred. share	Coverage	Clipped
KION						
β = 0.3: value model	0.300	0.080	0.1043	0.553	0.436	0.0%
β = 0.3: leave-one-out	0.300	0.547	0.0852	0.612	0.342	0.0%
β = 0.1: value model	0.100	0.590	0.0714	0.655	0.287	0.0%
β = 0.1: leave-one-out	0.100	4.393	0.0009	0.726	0.011	0.0%
KL budget 0.1: value model	0.254	0.104	0.1018	0.564	0.427	0.0%
KL budget 0.1: leave-one-out	0.740	0.099	0.1048	0.553	0.442	0.0%
KL budget 0.5: value model	0.112	0.556	0.0789	0.630	0.304	0.0%
KL budget 0.5: leave-one-out	0.313	0.501	0.0859	0.611	0.338	0.0%
β = 0.3: value model, learning rate ×10	0.300	0.176	0.1062	0.547	0.350	0.3%
Yambda						
β = 0.1: value model	0.100	0.102	0.1158	0.617	0.479	0.0%
β = 0.1: leave-one-out	0.100	1.540	0.0491	0.755	0.204	0.0%
β = 0.03: value model	0.030	4.391	0.0032	0.774	0.015	1.1%
β = 0.03: leave-one-out	0.030	7.272	0.0007	0.781	0.003	3.3%
KL budget 0.2: value model	0.066	0.189	0.1138	0.625	0.454	0.0%
KL budget 0.2: leave-one-out	0.215	0.200	0.1099	0.632	0.450	0.0%
KL budget 0.5: value model	0.067	0.490	0.0951	0.665	0.384	0.0%
KL budget 0.5: leave-one-out	0.151	0.494	0.0939	0.664	0.396	0.0%

compares lists for the same history, a value-model advantage with an imperfect critic still carries noise between histories, which averages out over a batch. We did not test this reading; the practical point does not depend on it. An anchor weight is not a quantity that can be compared between estimators, rewards, datasets or papers, and a KL budget is.

Clipping hardly engaged. The share of sampled items beyond the clipping threshold is at most 0.3% in every run that did not collapse, the run with ten times the learning rate of the policy included; only collapsing runs reach a few percent. The small learning rate and the anchor keep almost every update inside the threshold.

5.4 Sequence credit and simulated feedback

At an equal distance, myopic credit is no worse. At a fixed weight the ablated runs are not comparable with the full one: with $\gamma = 0$ the policy ends 31 times further from the base recommender (Table 7, last

Table 7: Credit ablations on the default reward at equal KL budgets and, for KION, at an equal anchor weight.

	β at end	KL	NDCG@10	on liked	Pred. share	Coverage
KION, KL budget 0.1						
whole rest of the list	0.254	0.104	0.1018	0.1217	0.564	0.427
own reward only ($\gamma = 0$)	1.830	0.100	0.1084	0.1275	0.549	0.426
one item per episode	1.597	0.102	0.1049	0.1263	0.552	0.439
KION, KL budget 0.5						
whole rest of the list	0.112	0.556	0.0789	0.0993	0.630	0.304
own reward only ($\gamma = 0$)	0.760	0.502	0.0920	0.1171	0.608	0.254
one item per episode	0.614	0.504	0.0944	0.1208	0.603	0.279
KION, fixed $\beta = 0.3$						
whole rest of the list	0.300	0.080	0.1043	0.1233	0.553	0.436
own reward only ($\gamma = 0$)	0.300	2.469	0.0557	0.0744	0.739	0.038
one item per episode	0.300	1.633	0.0776	0.1026	0.660	0.075
Yambda, KL budget 0.2						
whole rest of the list	0.066	0.189	0.1138	0.0993	0.625	0.454
own reward only ($\gamma = 0$)	0.267	0.203	0.1155	0.1019	0.624	0.435
one item per episode	0.153	0.213	0.1173	0.1033	0.621	0.423
Yambda, KL budget 0.5						
whole rest of the list	0.067	0.490	0.0951	0.0836	0.665	0.384
own reward only ($\gamma = 0$)	0.165	0.499	0.1075	0.0957	0.633	0.367
one item per episode	0.093	0.484	0.1104	0.0980	0.625	0.361

KION panel), presumably because per-item rewards standardise to larger advantages than returns that share most of their terms. At an equal KL budget the comparison is fair. On KION a policy credited with each item's own reward only is 6% more relevant than the policy credited with the whole rest of the list at a predicted share 1.5 points lower, which puts it on or slightly above the same frontier (Figure 2); one-item episodes give the same picture, with lower coverage at the larger budget, and the second session reproduces these numbers to the third decimal. On Yambda the picture is the same at the budget of 0.2 (NDCG@10 +1.5% and +3.1% at nearly the same predicted share, with lower coverage), and at 0.5 both ablated policies lie close to the line between the full-credit runs. We find no sign that crediting an item with what follows it, or learning from generated contexts, contributes to the result.

Sampling the simulated response changes the scale, not the policy. The reward of the k-th item depends on the responses assumed for the items before it. Fine-tuning with a response sampled from the predicted probability, instead of its expectation, gives nearly the same policy (Table 16 in the appendix): NDCG@10 0.103 against 0.104 on KION at $\beta = 0.3$, 0.073 against 0.071 on KION and 0.118 against 0.116 on Yambda at $\beta = 0.1$, where the sampled-feedback runs also ended somewhat nearer to the base recommender. What the assumption changes is the level of every number: the same policy has a predicted share of 0.55, 0.62 or 0.69 on KION, and 0.62, 0.73 or 0.87 on Yambda, depending on whether earlier items are assumed to have received the expected response, a sampled one, or to have been finished. A multi-step predicted reward is in part a property of the simulation, and only differences between policies under one assumption are interpretable.

5.5 Whose reward is it

The policy gradient learns items, not users. On KION 99% of the gain of the main run is what the held-out model predicts for the same lists shown to other users; at a weaker anchor the personal share is negative (Table 8). On Yambda the small gain is almost entirely impersonal as well. Re-ranking the base recommender's candidates by the same reward model has a personal share of 9% on KION and 44% on Yambda: the reward model does know something about the user, and a method that asks it about each candidate uses that knowledge, while the policy gradient does not find it.

A table of item means reaches the same point. Re-ranking candidates by $\bar{\rho}(j) + w \log \pi_0(j \mid h)$, where $\bar{\rho}(j)$ is the mean consumed share of item j in the training period, uses no reward model and no RL. At $w = 0.3$ on KION it reaches a predicted share of 0.549 at NDCG@10 0.109; the main RL run reaches 0.553 at 0.105. What RL extracts from the reward model on this reward is, to a first approximation, a ranking of items by how often they are finished.

5.6 Does it take RL?

Re-ranking reaches the frontier of RL on KION. At nearly the same predicted share as the main RL run (0.545 against 0.553), re-ranking with $w = 0.3$ has NDCG@10 0.113 against 0.105, coverage 0.53 against 0.45 and a logged share per recommended item of 0.0201 against 0.0174 (Table 9, Figure 3). Most of this difference is a head start: re-ranking begins from the ranked lists of the base recommender and RL from its generated lists, which are 4% less relevant and cover less of the catalogue. Measured from its own starting point, each method pays about the same relevance for the same predicted share (−7.6% of

Table 8: The gain in predicted share (held-out model) and the part of it that other users would give the same lists. "Personal" is the share of the gain that depends on who receives the list.

	NDCG@10	Own user	Another user	Gain	Gain, another	Personal
KION						
Base recommender	0.1173	0.455	0.442	0.000	0.000	–
RL, β = 0.3 (main run)	0.1047	0.553	0.539	0.098	0.097	1%
RL, β = 0.1	0.0714	0.655	0.647	0.200	0.205	−3%
Re-ranking, reward model, w = 0.3	0.1127	0.545	0.523	0.090	0.081	9%
Re-ranking, item mean, w = 0.3	0.1088	0.549	0.535	0.094	0.093	1%
Yambda						
Base recommender	0.1172	0.599	0.498	0.000	0.000	–
RL, β = 0.3 (main run)	0.1183	0.609	0.508	0.010	0.010	−1%
RL, β = 0.1	0.1158	0.617	0.514	0.018	0.017	7%
Re-ranking, reward model, w = 0.3	0.1155	0.619	0.509	0.019	0.011	44%
Re-ranking, item mean, w = 0.3	0.1136	0.621	0.518	0.021	0.020	4%

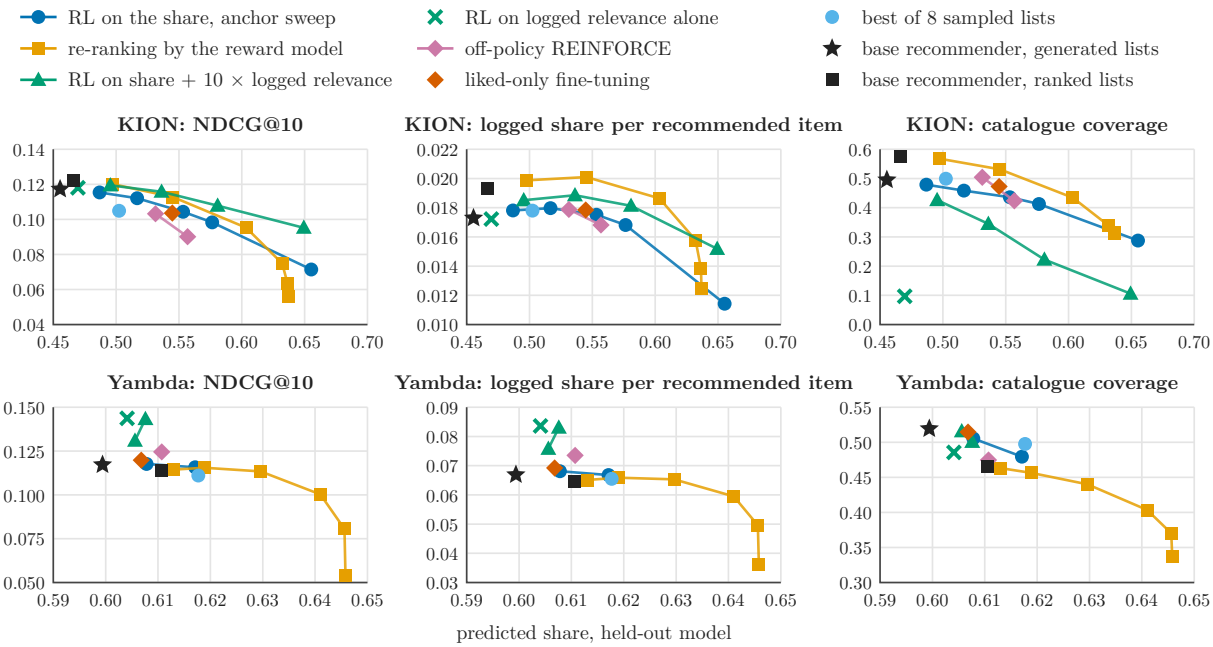


Figure 3: Methods compared at the predicted share they reach (held-out model): relevance, the logged share per recommended item, and coverage. Points of a line are one method at several strengths. RL and the fine-tuning methods start from the generated lists of the base recommender, re-ranking from its ranked lists.

NDCG@10 for +0.08 of share by re-ranking, −10.7% for +0.10 by RL). The logged share is the one axis on which re-ranking moves and RL does not: +4% from its starting point against +0.6%. Liked-only fine-tuning and off-policy REINFORCE, which never consult the reward model, land next to the main RL run in predicted share and relevance. No comparison favours on-policy RL, and re-ranking needs no training.

On Yambda the log-based methods lead. Off-policy REINFORCE on logged sessions raises NDCG@10 by 6% and the logged share by 10% at the predicted share that RL reaches, through repeats; relevance on new tracks is unchanged (Table 11). Re-ranking buys more predicted share for a small loss of relevance. No method moves the predicted share by more than a few points without leaving the region where the lists are still relevant.

Table 9: Ways of raising the reward without on-policy RL. Re-ranking starts from the ranked lists of the base recommender, every other method from its generated lists.

	NDCG@10	on liked	above usual	Pred. share	Logged	Surplus	Coverage
KION							
Base recommender	0.1173	0.1214	0.1261	0.455	0.0173	0.0012	0.495
Base recommender, ranked	0.1220	0.1293	0.1326	0.466	0.0193	0.0014	0.574
RL, $\beta = 0.3$ (main run)	0.1047	0.1229	0.1142	0.553	0.0174	0.0005	0.449
Liked-only fine-tuning	0.1036	0.1258	0.1134	0.545	0.0178	0.0003	0.473
Off-policy REINFORCE, $\varkappa = 0.5$	0.1032	0.1237	0.1127	0.531	0.0179	0.0005	0.504
Re-ranking, $w = 1$	0.1201	0.1334	0.1315	0.497	0.0199	0.0013	0.568
Re-ranking, $w = 0.3$	0.1127	0.1344	0.1249	0.545	0.0201	0.0009	0.531
Re-ranking, $w = 0.1$	0.0953	0.1234	0.1080	0.603	0.0186	0.0003	0.434
Best of 8 sampled lists	0.1049	0.1125	0.1128	0.502	0.0178	0.0010	0.499
Yambda							
Base recommender	0.1172	0.1005	0.0998	0.599	0.0669	0.0033	0.519
Base recommender, ranked	0.1138	0.0989	0.0986	0.611	0.0645	0.0029	0.466
RL, $\beta = 0.3$ (main run)	0.1183	0.1029	0.1023	0.609	0.0686	0.0032	0.506
Liked-only fine-tuning	0.1198	0.1035	0.1028	0.607	0.0692	0.0036	0.515
Off-policy REINFORCE, $\varkappa = 0.5$	0.1245	0.1090	0.1083	0.611	0.0735	0.0039	0.475
Re-ranking, $w = 1$	0.1144	0.0998	0.0995	0.613	0.0651	0.0030	0.463
Re-ranking, $w = 0.3$	0.1155	0.1012	0.1008	0.619	0.0659	0.0029	0.456
Re-ranking, $w = 0.1$	0.1134	0.0998	0.0993	0.630	0.0653	0.0028	0.440
Best of 8 sampled lists	0.1110	0.0967	0.0961	0.618	0.0655	0.0030	0.498

Table 10: Rewards that an item cannot earn on its own account. "Rel. share" is the held-out model's prediction of the share relative to the item's usual (Equation 1); 0.5 is "as usual".

	NDCG@10	on liked	Pred. share	Rel. share	Logged	Surplus	Coverage
KION							
Base recommender	0.1173	0.1214	0.455	0.529	0.0173	0.0012	0.495
Share (the main reward), $\beta = 0.3$	0.1043	0.1233	0.553	0.525	0.0175	0.0005	0.436
Base recommender, ranked	0.1220	0.1293	0.466	0.528	0.0193	0.0014	0.574
Relative share, $\beta = 0.3$	0.1168	0.1190	0.434	0.541	0.0168	0.0013	0.481
Relative share, $\beta = 0.1$	0.1114	0.1111	0.401	0.554	0.0154	0.0014	0.439
Re-ranking by relative share, w = 0.1	0.1206	0.1263	0.437	0.546	0.0185	0.0020	0.552
Share + 10 × logged relevance, $\beta = 0.3$	0.1194	0.1303	0.495	0.535	0.0185	0.0011	0.426
Share + 10 × logged relevance, $\beta = 0.1$	0.1158	0.1323	0.536	0.540	0.0188	0.0008	0.345
Logged relevance alone, $\beta = 0$	0.1180	0.1279	0.470	0.553	0.0172	0.0007	0.097
Yambda							
Base recommender	0.1172	0.1005	0.599	0.469	0.0669	0.0033	0.519
Share (the main reward), $\beta = 0.1$	0.1158	0.1008	0.617	0.465	0.0668	0.0029	0.479
Base recommender, ranked	0.1138	0.0989	0.611	0.473	0.0645	0.0029	0.466
Relative share, $\beta = 0.1$	0.1107	0.0960	0.595	0.487	0.0628	0.0040	0.476
Re-ranking by relative share, w = 0.1	0.1120	0.0977	0.610	0.486	0.0633	0.0034	0.444
Share + 10 × logged relevance, $\beta = 0.1$	0.1309	0.1139	0.606	0.468	0.0758	0.0035	0.516
Share + 10 × logged relevance, $\beta = 0$	0.1433	0.1245	0.608	0.467	0.0830	0.0037	0.501
Logged relevance alone, $\beta = 0$	0.1437	0.1259	0.604	0.468	0.0836	0.0039	0.486

A reward for the whole list does not change the picture. With a reward that adds the distance of each item to the items before it, an item's worth depends on its neighbours, and the argument that re-ranking suffices because the reward is a sum over items no longer applies. RL does optimise it: the list reward rises from 0.76 to 0.85 at $\beta = 0.3$ on KION, for 5% of NDCG@10 (Table 20 in the appendix). Greedy re-ranking with the same reward reaches 0.85 for 1.6% of the NDCG@10 of its starting point, and 0.93 at the relevance RL had. On Yambda RL at $\beta = 0.1$ raises the list reward from 0.96 to 0.98 without losing relevance, and greedy re-ranking reaches 1.00 while staying above the relevance of the ranked lists it starts from. A greedy construction is not optimal for this reward in principle; in practice it is ahead of the policy gradient.

5.7 Other rewards

A length-neutral reward removes the drift and most of the gain. Under the relative-share reward the policy no longer moves towards films: the mean duration of a recommended title rises from 5.7 to 6.7 hours where the share reward took it to 3.4, and relevance is kept (-0.4% at $\beta = 0.3$, Table 10). What is left to gain is small. The predicted relative share rises from 0.529 to 0.541, and the logged surplus, which the share reward halved, stays where it was (0.0013 against 0.0012). Re-ranking by the same signal gains more: a logged surplus of 0.0020 against 0.0014 for the ranked lists it starts from, for 1% of NDCG@10. On Yambda the reward raises the predicted relative share from 0.469 to 0.488 and the logged surplus from 0.0033 to 0.0040 for 5.5% of NDCG@10. This is the one place where RL moved a logged, length-neutral quantity more than re-ranking by the same signal did (0.0034). It rests on one run, and the interval of this quantity over the user sample alone is $\pm 13\%$ in the main run, so we do not build on it. Once the reward cannot be earned by choosing easy items, what remains is the personal part, and it is small.

Logged relevance is the reward term that helps, and on Yambda it helps by replaying. On KION the share plus ten times the logged-relevance term with $\beta = 0.3$ is the only RL configuration that improves on the base recommender in relevance, predicted share and logged share at once: NDCG@10 +1.8%, predicted share +8.8%, logged share +7%. The term protects hits and the anchor protects variety: without the anchor, coverage falls from 0.50 to 0.11 (Table 18). On Yambda logged relevance alone, with no anchor, raises NDCG@10 by 23%, which is the largest gain in relevance of the study. Table 11 shows what it consists of: relevance on repeats rises by a third, relevance on new tracks falls by 23%, and the share of recommendations taken from the history rises from 51% to 62%. The policy moves towards the replay baseline, which is what the reward pays for on a log where repeats dominate. It is a legitimate gain on this metric and should not be read as better recommendation of new music. With an anchor the move is milder: the share plus ten times logged relevance at $\beta = 0.1$ raises NDCG@10 by 12% and the logged share by 13%, through repeats (+15%), while relevance on new tracks falls by 4% and coverage is kept.

Table 11: Yambda: relevance on tracks from the visible history and on new tracks.

	NDCG@10	on repeats	on new	From history	Logged	Coverage
Replay of the history	0.1873	0.3722	0.0000	100%	0.1103	0.519
Base recommender	0.1172	0.1865	0.0261	51%	0.0669	0.519
RL on the share, $\beta = 0.3$ (main run)	0.1183	0.1868	0.0271	51%	0.0686	0.506
Liked-only fine-tuning	0.1198	0.1903	0.0269	51%	0.0692	0.515
Off-policy REINFORCE, $\varkappa = 0.5$	0.1245	0.1999	0.0261	49%	0.0735	0.475
Re-ranking, w = 0.3	0.1155	0.1786	0.0321	43%	0.0659	0.456
RL on logged relevance alone, $\beta = 0$	0.1437	0.2487	0.0201	62%	0.0836	0.486
RL on share + 10 × logged relevance, $\beta = 0.1$	0.1309	0.2142	0.0250	55%	0.0758	0.516

Retention and personal rewards. Rewarding the predicted return raises it by 3 to 7% on KION for a loss of 8 to 23% of NDCG@10 and does not move it on Yambda (Table 17 in the appendix); the signal varies four to ten times less between a user's candidates than between users, so there is little for a policy to choose. The session signal on KION is derived from calendar days and the order of rows within a day, and we do not interpret its reward. A reward that subtracts what the same model predicts for another user cannot collapse onto one list even without an anchor (pair overlap 0.33 against 0.98 on KION), and raises the personal part at $\beta = 0.1$ from 0.014 to 0.024 on KION for 3% of NDCG@10, and by 5% on Yambda for 0.5% (Table 19).

6 Discussion

What the instruments add. The usual defence against reward hacking is a second model, and here it fails in an instructive way: it catches the exploitation that is specific to one set of weights (0.84 against 0.69 without an anchor) and misses the exploitation that any model of this kind invites. Both reward models have learned, correctly, that some titles are finished more often than others, and a policy that serves those titles to everyone raises both. The instruments that expose this are cheap: score the same lists for another user, and count what the log says about the recommended items the user reached.

What RL added. For a reward that is a sum of per-item predictions and is mostly determined by the item, the optimum of the anchored objective is close to a re-ranking of π_0 by item quality, and a policy gradient is an expensive way to approximate it: a fine-tuning run costs minutes of GPU time per operating point, a re-ranking weight costs nothing. The estimator comparison and the credit ablations say the same from the inside: at an equal distance from π_0, nothing that distinguishes one policy-gradient method from another mattered. The list-dependent reward was the case built to favour sequential decisions, and a greedy heuristic still led.

Relation to the studies we build on. Petrov and Macdonald [8] keep relevance inside the reward; our results agree that this term is what keeps hits, and add that it keeps whatever is easiest to hit, so that an anchor or a diversity term has to keep the rest. Matveev et al. [9] learn from logged actions and logged feedback; on the log with a recommender flag and long sessions this was the best method in logged terms, and their warning about judging with the model that selected applies to our predicted shares in full.

Practical implications. Report the KL distance a fine-tuned policy ended at, not only the weight that was set. Before fine-tuning against a feedback model, re-rank by the same model and by item means: if the frontier is the same, the model's contribution is knowledge about items. Score lists for another user. Prefer a reward that an item cannot earn on its own account, and expect it to be worth less than the raw signal suggested.

7 Limitations

- **One seed per configuration.** The repeat of the KION session bounds hardware noise, not the variation between seeds; differences of a few thousandths of NDCG@10, such as those between credit assignments, need the seed runs.
- **Everything is offline.** Table 12 lists what this leaves open. The logged check sees only recommended items that users reached without the recommender's help.

Table 12: What the offline evidence settles and what needs an online experiment.

Claim	Offline evidence here	What an online experiment would add
Without an anchor the lists collapse	Conclusive: a fact about the policy, measured without any model of the user	Nothing
An anchor weight is not comparable, a KL distance is	Conclusive for this setup: a statement about the optimiser	Nothing
The predicted gain is impersonal	Conclusive as a statement about the reward model's predictions	Whether real responses to the fine-tuned lists depend on the user more than predicted
The predicted gain on KION is not a gain in consumed share	Suggestive: the logged share per recommended item is unchanged, but only for the items users reached on their own	The consumed share of served recommendations, the decisive measurement
Re-ranking is at least as good as RL	Under the same reward model and the same logged checks	Real responses; serving cost of scoring 100 candidates against one generation pass
Retention rewards buy little	Weak: return is a counterfactual outcome that the log shows for one policy	Return rates under each policy
Greedy construction suffices for a diversity reward	Conclusive for the modelled reward	Whether users value the diversity that is rewarded

- **The Yambda grids are narrower than KION's.** Sampled feedback, the list reward, and the retention and personal rewards were run at one anchor weight each, and the credit ablations at the two budgets only.
- **The base recommender on Yambda is below a replay heuristic,** and gains in relevance there are gains over this recommender.
- **The main run's weight is not a blind choice** (Section 4).
- **The explanation of why weights differ between estimators is a hypothesis** that no run isolates.
- **The personal part is the reward model's own belief** about how much of its prediction depends on the user.
- **Re-ranking and RL start from different lists,** ranked and generated; on KION this favours re-ranking by about 4%.
- **The distance in the list reward is measured in the base recommender's own embedding space,** which the greedy method and the evaluation share.
- **Baselines are untuned.**

8 Conclusion

We fine-tuned a generative recommender against a learned reward model in the setting with the fewest protections. The reward rose, and a model trained on other users confirmed it. The log did not: users consumed no more of what was recommended, and less of it relative to what is usual for the items. Other users would have given the same lists the same reward, and a table of item means reproduces the result. Making the reward neutral to the item removes the artefact together with most of the gain. On the side of optimisation, the anchor weight turned out not to be a quantity one can report or transfer, while the KL distance is; at an equal distance the choice of advantage estimator and of credit assignment made no difference we could detect, and PPO's clipping hardly acted. At every moderate operating point, re-ranking the pretrained recommender's candidates was at least as good as RL, including for a reward that depends on the whole list.

What moved logged outcomes came from the log: a logged-relevance term inside the reward, and off-policy learning from logged sessions. For a reward model to be worth optimising with RL, it has to know something about the user that cannot be looked up per item, and the reward has to be built so that only that knowledge pays. In our data that part was small. Whether it is small in general, and whether real users respond to the fine-tuned lists as the log suggests, are questions for more seeds and for an online experiment.

Code and data availability

The code, the configuration and every table of every session are in the repository `github.com/daniilantonovv/RecSysMaxxing` (`results/`); the tables of this paper are typeset from those files by `report/tables.py`.

References

[1] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[2] A. V. Petrov and C. Macdonald, "Generative Sequential Recommendation with GPTRec," *arXiv preprint arXiv:2306.11114*, 2023.

[3] S. Rajput *et al.*, "Recommender Systems with Generative Retrieval," *arXiv preprint arXiv:2305.05065*, 2023.

[4] J. Zhai *et al.*, "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," *arXiv preprint arXiv:2402.17152*, 2024.

[5] J. Deng *et al.*, "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[6] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. Chi, "Top-K Off-Policy Correction for a REINFORCE Recommender System," *arXiv preprint arXiv:1812.02353*, 2018.

[7] E. Ie *et al.*, "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology," *arXiv preprint arXiv:1905.12767*, 2019.

[8] A. Petrov and C. Macdonald, "Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning," *arXiv preprint arXiv:2403.04875*, 2024.

[9] A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, and K. Khrylchenko, "Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction," *arXiv preprint arXiv:2607.02818*, 2026.

[10] T. Yu *et al.*, "MOPO: Model-based Offline Policy Optimization," *arXiv preprint arXiv:2005.13239*, 2020.

[11] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-Based Offline Reinforcement Learning," *arXiv preprint arXiv:2005.05951*, 2020.

[12] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[13] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.

[14] C. Gao *et al.*, "Alleviating Matthew Effect of Offline Reinforcement Learning in Interactive Recommendation," *arXiv preprint arXiv:2307.04571*, 2023.

[15] C. Gao *et al.*, "CIRS: Bursting Filter Bubbles by Counterfactual Interactive Recommender System," *arXiv preprint arXiv:2204.01266*, 2022.

[16] R. Jiang, S. Chiappa, T. Lattimore, A. György, and P. Kohli, "Degenerate Feedback Loops in Recommender Systems," *arXiv preprint arXiv:1902.10730*, 2019.

[17] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019.

[18] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06939*, 2015.

[19] F. Sun *et al.*, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019.

[20] E. Botta *et al.*, "PinRec: Unified Generative Retrieval for Pinterest Recommender Systems," *arXiv preprint arXiv:2504.10507*, 2025.

[21] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Generative Slate Recommendation with Reinforcement Learning," *arXiv preprint arXiv:2301.08632*, 2023.

[22] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-Supervised Reinforcement Learning for Recommender Systems," *arXiv preprint arXiv:2006.05779*, 2020.

[23] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Supervised Advantage Actor-Critic for Recommender Systems," *arXiv preprint arXiv:2111.03474*, 2021.

[24] K. Christakopoulou *et al.*, "Reward Shaping for User Satisfaction in a REINFORCE Recommender," *arXiv preprint arXiv:2209.15166*, 2022.

[25] E. Ie *et al.*, "RecSim: A Configurable Simulation Platform for Recommender Systems," *arXiv preprint arXiv:1909.04847*, 2019.

[26] J.-C. Shi, Y. Yu, Q. Da, S.-Y. Chen, and A.-X. Zeng, "Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning," *arXiv preprint arXiv:1805.10000*, 2018.

[27] K. Zhao *et al.*, "KuaiSim: A Comprehensive Simulator for Recommender Systems," *arXiv preprint arXiv:2309.12645*, 2023.

[28] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to Trust Your Model: Model-Based Policy Optimization," *arXiv preprint arXiv:1906.08253*, 2019.

[29] T. Coste, U. Anwar, R. Kirk, and D. Krueger, "Reward Model Ensembles Help Mitigate Overoptimization," *arXiv preprint arXiv:2310.02743*, 2023.

[30] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust Region Policy Optimization," *arXiv preprint arXiv:1502.05477*, 2015.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[32] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, "Sequence Tutor: Conservative Fine-Tuning of Sequence Generation Models with KL-control," *arXiv preprint arXiv:1611.02796*, 2016.

[33] N. Jaques *et al.*, "Way Off-Policy Batch Deep Reinforcement Learning of Implicit Human Preferences in Dialog," *arXiv preprint arXiv:1907.00456*, 2019.

[34] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[35] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[36] Y. Wu, G. Tucker, and O. Nachum, "Behavior Regularized Offline Reinforcement Learning," *arXiv preprint arXiv:1911.11361*, 2019.

[37] R. Zhan *et al.*, "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation," *arXiv preprint arXiv:2206.06003*, 2022.

[38] H. Zhao, L. Zhang, J. Xu, G. Cai, Z. Dong, and J.-R. Wen, "Uncovering User Interest from Biased and Noised Watch Time in Video Recommendation," *arXiv preprint arXiv:2308.08120*, 2023.

[39] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems," *arXiv preprint arXiv:1902.05570*, 2019.

[40] Q. Cai *et al.*, "Reinforcing User Retention in a Billion Scale Short Video Recommender System," *arXiv preprint arXiv:2302.01724*, 2023.

[41] T. A. Mann *et al.*, "Learning from Delayed Outcomes via Proxies with Applications to Recommender Systems," *arXiv preprint arXiv:1807.09387*, 2018.

[42] Y. Su *et al.*, "Long-Term Value of Exploration: Measurements, Findings and Algorithms," *arXiv preprint arXiv:2305.07764*, 2023.

[43] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[44] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," *arXiv preprint arXiv:1506.03099*, 2015.

[45] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, "Sequence Level Training with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06732*, 2015.

[46] C. Guo *et al.*, "PROMISE: Process Reward Models Unlock Test-Time Scaling Laws in Generative Recommendations," *arXiv preprint arXiv:2601.04674*, 2026.

[47] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Offline Evaluation for Reinforcement Learning-based Recommendation: A Critical Issue and Some Alternatives," *arXiv preprint arXiv:2301.00993*, 2023.

[48] M. Dudik, J. Langford, and L. Li, "Doubly Robust Policy Evaluation and Learning," *arXiv preprint arXiv:1103.4601*, 2011.

[49] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches," *arXiv preprint arXiv:1907.06902*, 2019.

[50] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[51] R. J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[52] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE Samples, Get a Baseline for Free!," in *ICLR 2019 Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

[53] A. Ploshkin *et al.*, "Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking And Retrieval," *arXiv preprint arXiv:2505.22238*, 2025.

[54] A. Petrov, I. Safilo, D. Tikhonovich, and D. Ignatov, "MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation," *arXiv preprint arXiv:2209.00325*, 2022.

[55] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *arXiv preprint arXiv:1711.05101*, 2017.

A Configurations and cost

Table 13 lists what differs between the datasets; everything else is in `results/<session>/experiment_config.json`.

Table 13: What the sessions ran; for Yambda, the two sessions together. Fine-tuning: 512 lists per iteration, 4 gradient steps per batch, 2,000 iterations for the main run and 1,000 for every other run.

	KION	**Yambda-50M**
Catalogue	titles with at least 10 training events	50,000 most frequent tracks
Users	at least 5 training events	at least 50 training events
Window, dropout	48 events, 0.2	128 events, 0.1
Epochs: recommender / reward models	at most 30 / 15, early stopping	at most 14 / 8, early stopping
Session gap, return window	calendar day, 7 days	30 minutes, 1 day
Items of the history in a list	blocked	allowed
Anchor weights	0, 0.1, 0.2, 0.3, 0.5, 1	0, 0.03, 0.05, 0.1, 0.3
KL budgets	0.1, 0.5	0.2, 0.5
Leave-one-out at weights	0.1, 0.3	0.03, 0.1
Credit ablations	weights 0.1, 0.3; both budgets	both budgets
Relative-share reward at weights	0, 0.1, 0.3	0.03, 0.1
Logged relevance	alone at 0; with the share at 0, 0.03, 0.1, 0.3	alone at 0; with the share at 0, 0.1
Sampled feedback, list reward at weights	0.1, 0.3	0.1
Retention rewards	session, return, share + return at 0.1, 0.3	return at 0.1
Personal rewards at weights	0, 0.1, 0.3	0.1

Table 14: Wall-clock minutes by stage on one RTX PRO 6000 (96 GB). One fine-tuning run of 1,000 iterations takes 6.6 minutes on KION and 18.5 on Yambda, with its evaluation on test and validation requests. The Yambda continuation reused the base models, the baselines and the runs of the first session.

	KION	**Yambda, first session**	**Yambda, continuation**
Base recommender	4	93	0
Two reward models	5	19	1
Baselines	8	112	2
Main fine-tuning run	17	39	0
Anchor sweep	39	56	39
KL budgets	13	18	19
Advantage estimators	27	45	17
Sampled simulated feedback	21	0	20
Split of the gain, re-ranking	6	2	2
Alternatives to RL	6	15	1
Retention rewards	38	0	18
Logged relevance in the reward	57	18	37
Personal rewards	44	1	41
Relative-share reward	22	38	2
List reward	19	0	21
Credit ablations	39	27	28

Table 15: Selection of the anchor weight on validation requests: the highest predicted share among the runs within 5% of the base recommender's NDCG@10.

	NDCG@10, validation	Pred. share, validation	Within tolerance	Selected
KION				
Base recommender	0.1296	0.457	yes	no
$\beta = 1$	0.1268	0.488	yes	yes
$\beta = 0.5$	0.1223	0.519	no	no
$\beta = 0.3$	0.1133	0.556	no	no
$\beta = 0.2$	0.1072	0.578	no	no
$\beta = 0.1$	0.0766	0.656	no	no
$\beta = 0$	0.0004	0.689	no	no
Yambda				
Base recommender	0.1314	0.599	yes	no
$\beta = 0.3$	0.1314	0.608	yes	no
$\beta = 0.1$	0.1313	0.616	yes	no
$\beta = 0.05$	0.1273	0.628	yes	yes
$\beta = 0.03$	0.0056	0.772	no	no
$\beta = 0$	0.0015	0.809	no	no

Table 16: Fine-tuning with expected and with sampled simulated feedback, and the held-out predicted share of each policy under three assumptions about the response to earlier items of the list.

				Held-out share when earlier items are		
	KL	NDCG@10	Coverage	expected	sampled	finished
KION						
Expected feedback, $\beta = 0.3$	0.080	0.1043	0.436	0.553	0.617	0.695
Sampled feedback, $\beta = 0.3$	0.087	0.1033	0.442	0.554	0.627	0.706
Expected feedback, $\beta = 0.1$	0.590	0.0714	0.287	0.655	0.716	0.768
Sampled feedback, $\beta = 0.1$	0.475	0.0730	0.287	0.645	0.717	0.773
Yambda						
Expected feedback, $\beta = 0.1$	0.102	0.1158	0.479	0.617	0.728	0.873
Sampled feedback, $\beta = 0.1$	0.076	0.1178	0.489	0.614	0.728	0.871

B Further rewards

Table 17: Retention rewards. "Continues" and "returns" are the held-out model's predictions for the generated lists.

	NDCG@10	Pred. share	Continues	Returns	Coverage	Overlap
KION						
Base recommender	0.1173	0.455	0.345	0.718	0.495	0.209
Session, $\beta = 0.3$	0.0951	0.438	0.404	0.754	0.567	0.078
Session, $\beta = 0.1$	0.0443	0.297	0.545	0.764	0.526	0.106
Return, $\beta = 0.3$	0.1077	0.459	0.368	0.739	0.521	0.141
Return, $\beta = 0.1$	0.0903	0.466	0.392	0.765	0.544	0.093
Share + return, $\beta = 0.3$	0.1091	0.502	0.355	0.733	0.491	0.183
Share + return, $\beta = 0.1$	0.0943	0.546	0.372	0.756	0.477	0.167
Yambda						
Base recommender	0.1172	0.599	0.947	0.913	0.519	0.001
Return, $\beta = 0.1$	0.1158	0.600	0.947	0.913	0.516	0.002

Table 18: Logged relevance in the reward, alone and added to the share.

	KL	NDCG@10	on liked	Pred. share	Logged	Coverage	Overlap
KION							
Base recommender	0.000	0.1173	0.1214	0.455	0.0173	0.495	0.209
Logged relevance alone, $\beta = 0$	1.237	0.1180	0.1279	0.470	0.0172	0.097	0.518
Share + 10 × logged relevance, $\beta = 0.3$	0.080	0.1194	0.1303	0.495	0.0185	0.426	0.282
Share + 10 × logged relevance, $\beta = 0.1$	0.276	0.1158	0.1323	0.536	0.0188	0.345	0.334
Share + 10 × logged relevance, $\beta = 0.03$	0.728	0.1078	0.1305	0.581	0.0181	0.222	0.428
Share + 10 × logged relevance, $\beta = 0$	1.914	0.0952	0.1203	0.649	0.0152	0.106	0.624
Share + 3 × logged relevance, $\beta = 0.3$	0.080	0.1130	0.1287	0.532	0.0184	0.421	0.269
Share + 3 × logged relevance, $\beta = 0.1$	0.311	0.1004	0.1241	0.595	0.0170	0.330	0.350
Share + 3 × logged relevance, $\beta = 0.03$	1.130	0.0854	0.1097	0.674	0.0134	0.162	0.518
Share + 3 × logged relevance, $\beta = 0$	2.059	0.0783	0.1020	0.680	0.0128	0.084	0.651
Yambda							
Base recommender	0.000	0.1172	0.1005	0.599	0.0669	0.519	0.001
Logged relevance alone, $\beta = 0$	1.012	0.1437	0.1259	0.604	0.0836	0.486	0.004
Share + 10 × logged relevance, $\beta = 0.1$	0.237	0.1309	0.1139	0.606	0.0758	0.516	0.002
Share + 10 × logged relevance, $\beta = 0$	0.978	0.1433	0.1245	0.608	0.0830	0.501	0.004

Table 19: Rewards with a baseline. "Personal part" is the predicted share for the user minus that for another user.

	KL	NDCG@10	Pred. share	Personal part	Coverage	Overlap
KION						
Base recommender	0.000	0.1173	0.455	0.014	0.495	0.209
Personal share, $\beta = 0.3$	0.022	0.1161	0.455	0.019	0.487	0.205
Personal share, $\beta = 0.1$	0.097	0.1135	0.448	0.024	0.453	0.213
Personal share, $\beta = 0$	1.168	0.0686	0.313	0.029	0.336	0.332
Share above the item mean, $\beta = 0.3$	0.187	0.1066	0.407	0.021	0.406	0.225
Share above the item mean, $\beta = 0.1$	0.980	0.0613	0.368	0.026	0.239	0.262
Share above the item mean, $\beta = 0$	4.183	0.0327	0.424	0.002	0.038	0.797
Yambda						
Base recommender	0.000	0.1172	0.599	0.101	0.519	0.001
Personal share, $\beta = 0.1$	0.054	0.1166	0.603	0.106	0.516	0.002
Share above the item mean, $\beta = 0.1$	0.315	0.1056	0.590	0.109	0.437	0.002

Table 20: A list-dependent reward: predicted share plus the mean distance between the items of the list in the base recommender's embedding space. "List reward" is the reward per item as the held-out model sees it. Greedy re-ranking is maximal marginal relevance over 100 candidates.

	NDCG@10	Pred. share	Distance	List reward	Coverage
KION					
Base recommender	0.1173	0.455	0.617	0.764	0.495
RL on the list reward, $\beta = 0.3$	0.1111	0.520	0.655	0.848	0.455
RL on the list reward, $\beta = 0.1$	0.0991	0.583	0.699	0.932	0.394
Greedy re-ranking, w = 1	0.1201	0.499	0.692	0.845	0.580
Greedy re-ranking, w = 0.3	0.1104	0.550	0.749	0.925	0.571
Greedy re-ranking, w = 0.1	0.0899	0.601	0.829	1.015	0.518
Best of 8 sampled lists	0.1060	0.495	0.665	0.828	0.511
RL on the share alone, $\beta = 0.3$	0.1043	0.553	0.636	0.871	0.436
RL on the share alone, $\beta = 0.1$	0.0714	0.655	0.622	0.966	0.287
Yambda					
Base recommender	0.1172	0.599	0.715	0.957	0.519
RL on the list reward, $\beta = 0.1$	0.1182	0.611	0.732	0.976	0.509
Greedy re-ranking, w = 1	0.1145	0.613	0.744	0.984	0.465
Greedy re-ranking, w = 0.3	0.1155	0.617	0.769	1.002	0.462
Greedy re-ranking, w = 0.1	0.1105	0.624	0.821	1.034	0.447
Best of 8 sampled lists	0.1147	0.611	0.761	0.991	0.516
RL on the share alone, $\beta = 0.3$	0.1176	0.608	0.706	0.961	0.505
RL on the share alone, $\beta = 0.1$	0.1158	0.617	0.694	0.964	0.479

C Further figures

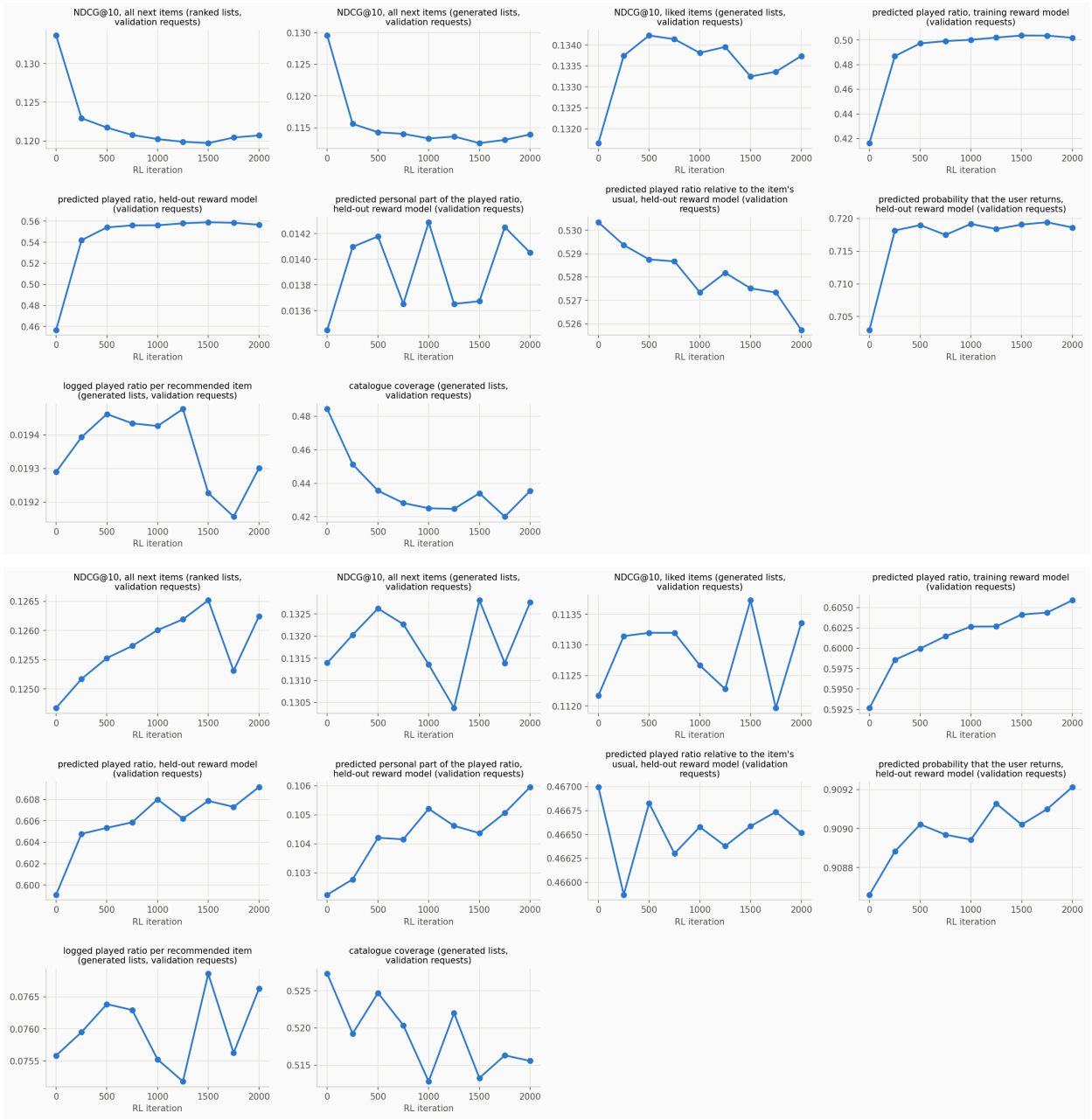


Figure 4: The main run on validation requests during fine-tuning: KION (top) and Yambda (bottom).

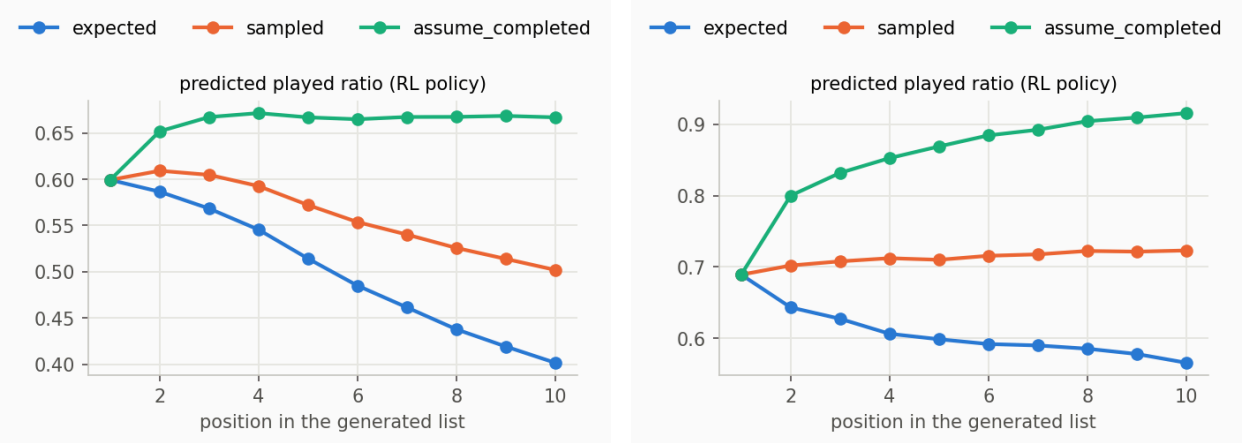


Figure 5: Predicted share by position of the list for the base recommender under three assumptions about the response to the items before it: KION (left) and Yambda (right).

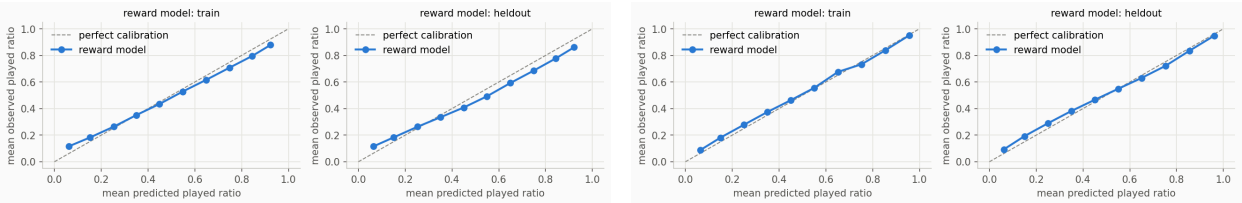


Figure 6: Calibration of the two reward models on test events: KION (left) and Yambda (right).

D Earlier iterations of the study

The pipeline was completed seven times before the sessions reported here (version 2 of this draft describes those runs in full). They differ from the present sessions in the reward models, which had fewer heads, and in the instruments available, and they agree with them where they overlap. The main run on KION was made three times with the same base recommender and other reward models: NDCG@10 changed by -11.4%, -11.4% and -11.1% and the held-out predicted share by $+22.8\%$, $+23.1\%$ and $+23.1\%$, against -10.7% and $+21.5\%$ here. On Yambda two earlier runs gave $+1.1\%$ and $+1.8\%$ of predicted share, against $+1.7\%$ here. The first two runs used REINFORCE with a leave-one-out baseline on Yambda, where the policy held at $\beta = 0.3$ and collapsed at $\beta = 0.1$, while the value-model runs held at 0.1 and collapsed at 0.03. The comparison was confounded by the learning rate and the number of lists per history; Table 6 repeats it with those held equal and confirms that the estimator alone moves the transition. Two findings of the earlier runs are not repeated here: a choice head trained to tell logged items from popularity-distributed replacements did not work as a relevance signal inside the reward, and a reward weighted by item duration moved the policy along the duration axis in the direction the normalisation favoured.